# Práctica de Integración Hardware-Software-Firmware (HSF)

Segundo corte — Integración de Sistemas de Información 2026-2

Universidad Santo Tomás — Facultad de Ingeniería en Tecnologías de la Información y las Comunicaciones

Programa: Ingeniería en Informática

Estudiante: [NombreEstudiante] — Código: [Codigo]

Entrega: Segundo corte, práctica de integración HSF

Fecha: [dd de mes de 2026]

Repositorio: https://github.com/Facabore/practica-hsf-segundo-corte

## 1. Introducción y contexto

Este documento sustenta el prototipo entregado para el Anexo 3 de la evaluación del segundo corte: un sistema que recorre las cuatro capas de un sistema informático — hardware, firmware, sistema operativo y software de aplicación — capturando datos de un dispositivo, procesándolos y generando información útil para la toma de decisiones.

No se dispuso de una placa física para esta práctica, por lo que el firmware se simula íntegramente en software (decisión justificada en la sección 3), siguiendo explícitamente una de las alternativas que la guía habilita: "cuando no se disponga de una placa física, el firmware puede simularse mediante un microcontrolador virtual, un script que emule telemetría o una fuente de datos controlada".

Todo el código, las pruebas y la evidencia citados en este documento están en el repositorio de GitHub referenciado arriba, y son reproducibles ejecutando los comandos que se indican en README.md.

## 2. Arquitectura de la solución

La arquitectura sigue el flujo recomendado por la guía: Hardware → Firmware → Comunicación → Sistema operativo → Python → Persistencia/Reporte → Usuario.

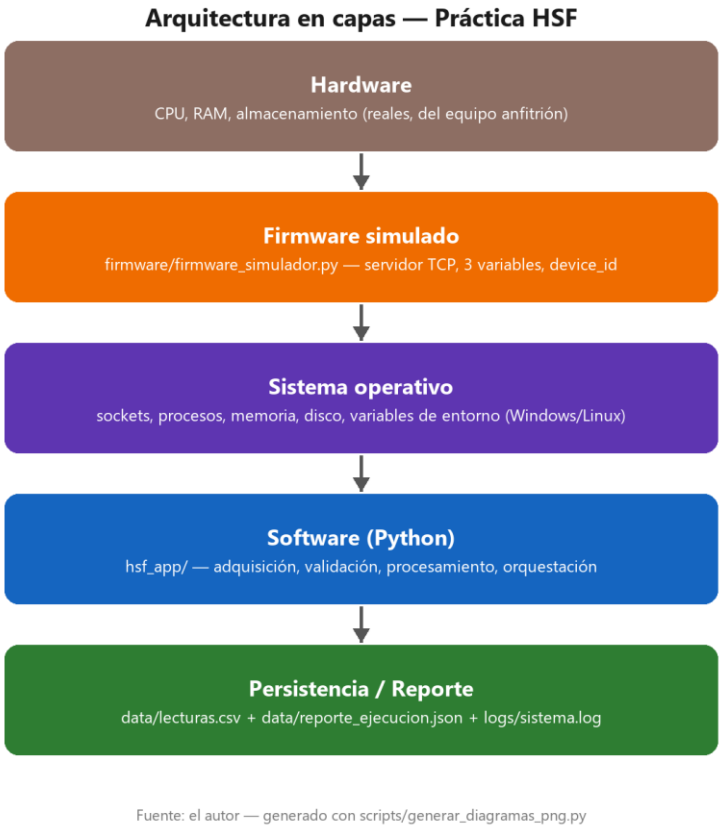



*Figura 1. Arquitectura en capas del prototipo. Fuente: el autor.*

| Capa | Responsabilidad en este prototipo | Archivo |
|---|---|---|
| Hardware | CPU, RAM y disco reales del equipo donde corre todo el prototipo | — (recursos del SO anfitrión) |
| Firmware | Genera telemetría de 3 variables con identificador de dispositivo cada 1 s, e inyecta una anomalía cada 7 ciclos | firmware/firmware_simulador.py |
| Comunicación | Socket TCP (127.0.0.1:8765), un mensaje JSON por línea | hsf_app/acquisition.py |
| Sistema operativo | Expone sockets, procesos, memoria, disco y variables de entorno; la app los consulta directamente | hsf_app/os_resources.py |
| Software | Valida, clasifica, orquesta el ciclo completo y expone el estado de la integración | hsf_app/*.py |
| Persistencia/Reporte | CSV de lecturas + reporte JSON con métricas agregadas | hsf_app/persistence.py, data/ |
| Presentación | Dashboard en vivo: vista general de la flota y una pestaña por equipo, con indicadores, recursos del SO, eventos y fallos controlados | hsf_app/dashboard.py, flota.py, web/ |

El flujo de una lectura individual, con el punto exacto donde cada tipo de error se intercepta sin detener la aplicación, se muestra a continuación:

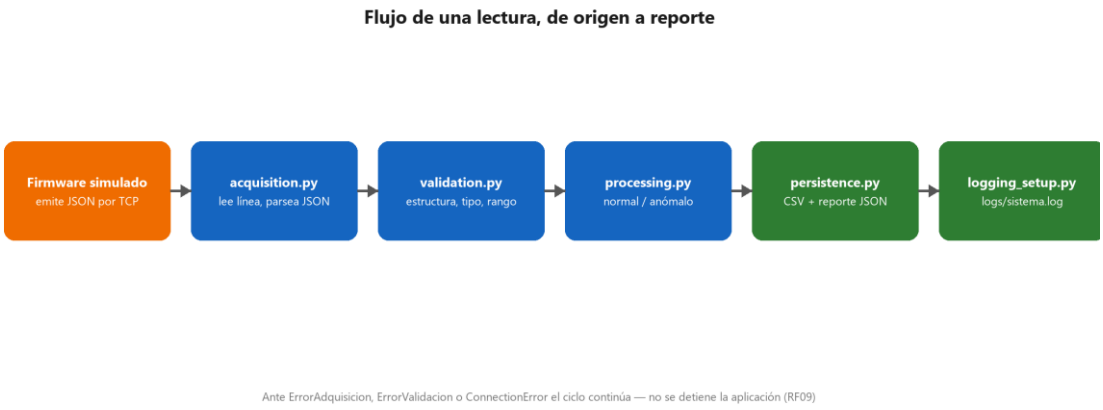



*Figura 2. Flujo de una lectura, de origen a reporte. Fuente: el autor.*

## 3. Decisiones técnicas y justificación

**Firmware 100 % simulado en Python, sobre socket TCP.** Se evaluaron tres alternativas que ofrece la guía: hardware real (Arduino/ESP32/Raspberry Pi), Wokwi, y un script de telemetría. Se descartó el hardware real porque introduce una variable de falla externa al código el día de la grabación del video. Se descartó Wokwi porque depende de un servicio web de terceros y complica la reproducibilidad del docente al clonar el repositorio. El script de telemetría sobre TCP corre igual en cualquier máquina con Python, sin instalar nada adicional, y ejercita una capa de comunicación real basada en servicios del sistema operativo (sockets).

**Python modular, no un único script.** El paquete hsf_app/ separa adquisición, validación, procesamiento, recursos del SO, persistencia y orquestación en archivos distintos con una única responsabilidad cada uno. Esto permite probar cada pieza de lógica de forma aislada (TDD real) y responde directamente al requisito no funcional de mantenibilidad del Anexo 3.

**TDD real, no retroactivo.** Las 95 pruebas de tests/ se escribieron antes que su implementación correspondiente, confirmando en rojo antes de pasar a verde. Esta disciplina encontró un defecto real durante la construcción: antes de envolver registrar_lectura_csv en un try/except, una ruta de persistencia inaccesible tumbaba toda la aplicación — una violación directa del RF09. Se corrigió escribiendo primero la prueba que exigía el comportamiento correcto (test_ejecutar_continua_si_falla_la_persistencia).

**CSV para la serie de datos, JSON para el reporte.** El RF07 acepta CSV o JSON; se usan ambos en su rol natural: CSV para una serie temporal tabular fácil de abrir en cualquier hoja de cálculo, JSON para un resumen jerárquico (métricas + snapshot de recursos del SO) que no encaja bien en filas y columnas.

**Reconexión con backoff exponencial, no reintento inmediato.** Un reintento inmediato y constante satura la red o el proceso remoto si la caída es prolongada. El backoff creciente (1, 2, 4, 8 segundos) da tiempo a que el firmware se recupere sin convertir la aplicación en un bucle ocupado.

**Dashboard con la librería estándar, sin frameworks.** El panel en vivo usa http.server y gráficas SVG dibujadas a mano, sin Flask ni librerías de gráficos por CDN, así funciona sin internet el día de la demostración. El orquestador no conoce el dashboard: solo informa sus eventos a un objeto de estado compartido (patrón observador), y el servidor HTTP lee de ese objeto. Así la lógica de integración se sigue probando sin red.

**Publicado en la nube, con controles abiertos.** La demo corre como servicio systemd en un VPS Ubuntu, detrás de Caddy con HTTPS automático (Let's Encrypt), en https://integration-corte2.duckdns.org. Todo escucha en 127.0.0.1 y solo Caddy queda expuesto; el servicio usa un

usuario sin privilegios que solo puede escribir en data/ y logs/, con techo de 1 CPU y 400 MB porque el servidor es compartido. Por decisión del dueño del servidor, los botones de cada equipo (encender, apagar, reiniciar y fallos) funcionan sin clave. Siguen activas las listas cerradas de comandos y equipos, la cabecera propia contra CSRF, el control de Host contra DNS rebinding y el techo de CPU del servicio, que contiene a Saturar CPU para que no afecte a los demás sitios. Riesgo aceptado: cualquier visitante puede apagar un equipo, y cualquiera puede volver a encenderlo. Con --solo-lectura se vuelve a publicar sin controles. Detalle en deploy/DESPLIEGUE.md.

**Varios dispositivos: un orquestador por equipo, no un orquestador más complejo.** Para la demo multi-dispositivo no se tocó la lógica de integración: se corre una instancia del mismo OrquestadorHSF por equipo, cada una con su conexión, su estado y su CSV (un archivo por equipo evita escrituras concurrentes), y una capa de flota solo las reúne para la vista general. Las gráficas usan el tiempo real como eje, así que un equipo caído deja un hueco visible en vez de estirar su línea, y cada equipo conserva el mismo color en todas las vistas.

## 4. Trazabilidad de requisitos

| Requisito | Cómo se cumple | Evidencia |
|---|---|---|
| RF01 | Cada trama trae device_id (--device-id); cada firmware escribe su device_manifest_<id>.json | firmware_simulador.py |
| RF02 | El firmware emite cada 1 s; la app lee en bucle continuo | acquisition.py |
| RF03 | validar_lectura revisa los 5 campos, sus tipos y sus rangos físicos | validation.py + 6 pruebas |
| RF04 | clasificar_lectura compara contra umbrales operativos por variable | processing.py + 5 pruebas |
| RF05 | Logging a consola y a logs/sistema.log, con nivel, evento y resultado | logging_setup.py |
| RF06 | CPU, memoria, disco, PID del proceso y variables de entorno en cada reporte | os_resources.py + 5 pruebas |
| RF07 | data/lecturas.csv (serie) + data/reporte_ejecucion.json (resumen) | persistence.py + 3 pruebas |
| RF08 | Métricas agregadas + snapshot de recursos al cierre, y en cualquier momento con «Descargar reporte» del dashboard, en JSON o CSV | app.py (MetricasEjecucion) + dashboard.py (armar_reporte) + persistence.py (reporte_a_csv) |
| RF09 | Errores de adquisición, validación, persistencia y conexión se cuentan y se continúa | app.py + 10 pruebas + §6 |
| RF10 | Log de "integración activa" al conectar, estado final impreso al cierre | app.py (main) |

No funcionales: seguridad (no se ejecuta ningún comando de sistema ni se interpreta la entrada como código — solo se parsea como JSON y se valida campo por campo); confiabilidad (ver RF09); rendimiento (ver §7, latencia medida); mantenibilidad (ver §3); observabilidad (ver RF05); portabilidad (ver pregunta 9 en §8); escalabilidad (ver pregunta 6 en §8).

## 5. Resultados de pruebas

95 pruebas automáticas en tests/, todas en verde (pytest tests/ -v), cubriendo validación, clasificación, recursos del SO, adquisición (con sockets reales en hilo local), persistencia, el orquestador completo (incluyendo reconexión, fallas de persistencia y detención desde otro hilo), el servidor del dashboard con sus defensas, y el lanzador de la demo con el firmware como proceso real. La matriz completa, cruzando cada caso con su evidencia automática y manual, está en docs/Matriz_Pruebas.md. Construir la demo en vivo destapó tres defectos más, cada uno cubierto primero con una prueba que fallaba: un firmware "colgado" (conexión abierta, sin datos) tumbaba la aplicación; en Windows podían escuchar dos firmwares en el mismo puerto; y la CPU del dashboard marcaba siempre 0 %. El detalle está en docs/Fallos_Controlados.md.

Además de las pruebas automáticas, se ejecutó el sistema real end-to-end: firmware simulado y aplicación como dos procesos independientes, generando data/lecturas.csv,

data/reporte_ejecucion.json y logs/sistema.log con datos reales (no inventados) — 20 lecturas, 2 de ellas anómalas por diseño del simulador.

## 6. Fallos controlados

Se ejecutaron los 6 fallos que pide el Anexo 3 §11 contra el sistema real (no simulados a mano): desconexión permanente de la fuente, dato con formato incorrecto, ruta de persistencia sin acceso, variable fuera de rango, saturación de CPU, e interrupción temporal de la red con recuperación. El detalle completo — causa, síntoma, detección, respuesta, evidencia real de consola y mejora propuesta para cada uno — está en docs/Fallos_Controlados.md. El hallazgo más relevante: el fallo de "ruta sin permisos" expuso durante la construcción un punto real donde la aplicación no era tolerante a fallas (RF09), corregido antes de la entrega gracias a TDD (ver §3).

## 7. Indicadores técnicos (Anexo 3 §12)

| Indicador | Cómo se midió | Meta | Resultado real |
|---|---|---|---|
| Disponibilidad de comunicación | lecturas recibidas / intentos, corrida con desconexión real (fallo 6) | ≥ 95 % | 40/41 ≈ 97.6 % |
| Latencia | diferencia timestamp firmware vs. recepción, 15 lecturas (loopback) | definir por caso | prom. 0.254 ms, máx. 0.375 ms |
| Datos válidos | lecturas válidas / recibidas, agregado de todas las corridas reales | ≥ 98 % | 63/64 ≈ 98.4 % |
| Tasa de errores | errores / eventos totales, agregado de todas las corridas reales | ≤ 5 % | 6.1 % agregado (0 % en la corrida base) |
| Tiempo de recuperación | fin de desconexión real → primera lectura normal (fallo 6) | definir por caso | 3.57 s |

La tasa de errores agregada por encima del 5 % es intencional: dos de las cinco corridas que alimentan este promedio son, por diseño, corridas de fallos controlados (§6), no operación normal. En operación normal (corrida base, sin fallos inyectados) la tasa de errores fue 0 %.

## 8. Preguntas de integración

**¿Qué responsabilidad pertenece al hardware y cuál al firmware? ¿Dónde se produce realmente cada transformación de datos?** En este prototipo, "hardware" es el equipo anfitrión real (CPU, memoria, disco) sobre el que corre todo el proceso; su responsabilidad es ejecutar instrucciones y ofrecer recursos. El "firmware" (firmware_simulador.py) decide qué dato generar, con qué forma y cada cuánto. La generación del valor crudo ocurre en el firmware; su serialización a JSON y envío por el socket, en la frontera firmware-comunicación; la deserialización, validación y clasificación, en hsf_app; la persistencia final, en la frontera software-sistema operativo.

**¿Qué servicios del sistema operativo utiliza la aplicación Python?** Sockets TCP (comunicación), el sistema de archivos (lectura/escritura de CSV, JSON y logs, con creación de directorios), y consultas de CPU, memoria, disco, PID del proceso y variables de entorno a través de psutil y los módulos estándar os/platform.

**¿Qué ocurre si el sistema operativo revoca permisos sobre un recurso utilizado por la aplicación?** Se demostró en el fallo 3 (§6): la llamada de escritura lanza OSError, el try/except en app.py la captura, se cuenta como errores_persistencia, se registra en el log, y el ciclo principal continúa sin perder el control.

**¿Qué ventajas y limitaciones tiene usar Python frente a C/C++ o Rust en una integración con dispositivos?** Ventajas: velocidad de desarrollo, manejo de excepciones expresivo (clave para RF09), un ecosistema rico para validación, persistencia y pruebas, y portabilidad casi directa entre Windows

y Linux. Limitaciones: el GIL impide paralelismo real de CPU dentro de un mismo proceso (se evidenció al construir el fallo 5, donde hubo que usar multiprocessing, no hilos); el rendimiento por operación es menor que C/C++/Rust; y Python no es apto para programar el firmware de un microcontrolador real de gama baja, donde C/C++ o Rust siguen siendo el estándar.

**¿Qué parte del sistema debería ejecutarse en firmware y cuál en software de aplicación? Justifique.** Todo lo que exige temporización estricta y bajo nivel pertenece al firmware, porque ahí la latencia y el determinismo importan más que la expresividad. Todo lo que exige lógica de negocio, validación compleja, persistencia estructurada y observabilidad pertenece al software de aplicación, porque ahí la mantenibilidad y la velocidad de iteración importan más. Mezclar ambas cosas en una sola capa dificulta probar y mantener el sistema, como demuestra la separación en módulos de este prototipo.

**¿Cómo cambiaría la arquitectura si se conectaran 100 dispositivos simultáneamente?** El primer paso ya está implementado y probado: la demo levanta tres equipos a la vez (compresor, bomba de agua y motor de banda), cada uno con su proceso de firmware, puerto y ritmo de anomalías; la aplicación abre una conexión por equipo, cada una en su hilo, y una capa de flota suma las métricas para la vista general. Detener un equipo no afecta a los demás: con SIM-002 caído, SIM-001 y SIM-003 siguieron de 76 a 79 lecturas. Para llegar a 100 harían falta tres cambios más: un servidor asíncrono (asyncio) en lugar de hilos; invertir la conexión, para que cada dispositivo se conecte a la aplicación e indique su device_id, como los equipos IoT reales detrás de NAT; y probablemente una cola de mensajes como MQTT, para desacoplar la ingestión del procesamiento.

**¿Qué riesgos de seguridad aparecen al recibir datos externos o ejecutar comandos del sistema?** El riesgo principal es tratar el dato recibido como código o como una ruta/comando sin sanear — por eso acquisition.py solo pasa la línea cruda por json.loads (nunca por eval ni por un shell), y validation.py rechaza cualquier estructura que no coincida con el esquema esperado. Un segundo riesgo es la denegación de servicio por datos malformados repetidos; se mitiga contando errores_adquisicion sin abortar el proceso. El dashboard, que sí dispara acciones, escucha solo en 127.0.0.1, acepta solo una lista cerrada de comandos (el nombre de la URL se busca, nunca se ejecuta), exige una cabecera propia que un sitio ajeno no puede enviar sin permiso CORS, y rechaza cualquier Host que no sea localhost (contra DNS rebinding).

**¿Cómo garantizaría integridad y trazabilidad de los datos?** Cada lectura persistida queda con su timestamp de origen y su estado (normal/anómalo) en el mismo renglón del CSV; cada evento relevante queda en logs/sistema.log con fecha/hora y nivel. En producción se añadiría un hash o firma por lote de lecturas y un identificador de sesión de ejecución, para demostrar que el archivo no fue alterado después de generado.

**¿Qué diferencias podrían presentarse entre Windows y Linux?** La más seria se encontró durante la práctica: con SO_REUSEADDR, Windows deja que un segundo firmware escuche el mismo puerto que otro ya activo (las conexiones van a cualquiera de los dos), mientras que en Linux esa opción solo evita esperar el TIME_WAIT; ahora se activa solo fuera de Windows, con una prueba que lo verifica. Además, los mensajes de error de socket difieren textualmente (WinError 10054/10061 en Windows frente a ConnectionResetError/ConnectionRefusedError en Linux, aunque ambos se capturan igual porque Python los normaliza a la misma jerarquía de excepciones); y el manejo de procesos en segundo plano cambia (por eso existen scripts/ejecutar_demo.ps1 y .sh por separado). psutil abstrae la mayoría de las diferencias de bajo nivel.

**¿Cómo diseñaría una estrategia de actualización del firmware sin afectar el servicio?** Con dos versiones del firmware corriendo en paralelo (azul-verde): la nueva versión se levanta en un canal distinto, se valida con tráfico de prueba, y solo entonces la aplicación conmuta su conexión hacia ella; la versión anterior se mantiene disponible por si hay que revertir.

**¿Qué pruebas adicionales aplicaría antes de llevar el prototipo a producción?** Pruebas de carga sostenida (horas, para detectar fugas de memoria o descriptores sin cerrar); pruebas de seguridad (fuzzing del parser JSON); pruebas de concurrencia real con múltiples dispositivos; y pruebas de recuperación ante caída del propio proceso de la aplicación, no solo del firmware.

**¿Qué componente sería el principal punto de falla y cómo lo convertiría en un componente tolerante a fallos?** El socket hacia cada firmware, como demuestra el fallo 1: si cae y no vuelve, ese equipo deja de enviar datos. Con varios equipos la caída queda aislada a uno, pero la aplicación sigue siendo un único proceso. Para tolerarlo, se introduciría redundancia (más de una fuente de datos o failover automático) y se separaría la adquisición de la persistencia mediante una cola intermedia, para que una caída de la fuente no bloquee el resto del pipeline.

## 9. Conclusiones

El prototipo demuestra, con evidencia real y no simulada a mano, el recorrido completo que exige la guía: desde la generación de un dato en una capa de firmware hasta su persistencia y reporte, pasando por servicios reales del sistema operativo anfitrión. La disciplina de TDD no fue un formalismo: encontró un defecto de tolerancia a fallas real antes de la entrega. Los 6 fallos controlados, ejecutados contra procesos reales, demuestran que el sistema cumple el RF09 en la práctica: ninguno de los 6 escenarios tumbó la aplicación.

La integración ya funciona con tres dispositivos en paralelo, aislados entre sí ante fallas. La limitación que queda es la escala: un hilo por equipo sirve para decenas, no para cientos. La pregunta de la sección 8 sobre 100 dispositivos señala dónde iría el siguiente esfuerzo de ingeniería: un servidor asíncrono, invertir la conexión y MQTT.

## 10. Referencias (APA 7)

Martinez Barrera, C. (2026). Evaluación segundo corte — Integración de Sistemas de Información 2026-2 [Guía de práctica]. Universidad Santo Tomás, Facultad de Ingeniería en Tecnologías de la Información y las Comunicaciones.

Python Software Foundation. (2026). Python 3 documentation — socket, json, logging, multiprocessing. https://docs.python.org/3/

Rodola, G. (2026). psutil documentation. https://psutil.readthedocs.io/

Microsoft. (2026). Windows Sockets error codes. Microsoft Learn. https://learn.microsoft.com/windows/win32/winsock/windows-sockets-error-codes-2